DAY 7-MCQ

Controlled components: form inputs bound to state

1.Which of the following best defines a *controlled component* in React?

A. A component that stores form data directly in the DOM.
B. A component whose form input values are handled by React state.
C. A component that does not require an onChange event handler.
D. A component that automatically syncs values without any state.

✅ **Answer:** B

2.In a controlled input field, which of the following causes the UI to update?

A. The user typing directly into the input field
B. The setState or useState state update triggered by onChange
C. The DOM automatically reflecting new input values
D. The browser's default input behavior

✅ **Answer:** B

3.If a text input uses value={stateValue} but no onChange handler is provided, what happens?

A. The input behaves as a controlled component.
B. The input is editable but not synced to state.
C. The input becomes read-only.
D. React automatically binds an onChange internally.

✅ **Answer:** C

4.Given this snippet:

```
const [name, setName] = useState("");

return <input value={name} onChange={(e) => setName(e.target.value)} />;
```

What is the role of onChange here?

A. Updates the DOM directly
B. Synchronizes input value changes to React state
C. Prevents default input events
D. Re-renders only on form submit

✅ **Answer:** B

5.Why might the following cause a React warning?

<input value={undefined} onChange={handleChange} />

A. Because the input mixes controlled and uncontrolled states
B. Because onChange is missing
C. Because value should always be a number
D. Because React forbids undefined props

✅ **Answer:** A

6.Which statement about controlled vs uncontrolled components is true?

A. Controlled components store input values in the DOM.
B. Uncontrolled components use React state for form data.
C. Controlled components give React full control over input values.
D. Uncontrolled components are faster and always preferred.

✅ **Answer:** C

7.What's the correct way to handle multiple form inputs in a single controlled form?

A. Create a separate state variable for each input without handlers.
B. Use one state object and update fields dynamically in onChange.
C. Directly mutate the DOM input values.
D. Use document.getElementById() for each input.

✅ **Answer:** B

8.What is the purpose of e.target.name in a controlled form handler?

A. To determine the HTML tag type
B. To get the form element's label
C. To identify which input's value changed
D. To reset the entire form

✅ **Answer:** C

9.What happens if value prop is omitted in an input but onChange is provided?

A. Input becomes controlled automatically
B. Input remains uncontrolled and won't reflect state updates

C. Input is editable and synced with React state
D. Input throws an error

✅ **Answer:** B

---

10. Given:

const [email, setEmail] = useState("test@example.com");

return <input defaultValue={email} onChange={(e) => setEmail(e.target.value)} />;

What is the component type and behavior?

A. Controlled; updates React state and value
B. Uncontrolled; defaultValue initializes but doesn't sync later
C. Semi-controlled; syncs partially on first render
D. Invalid; cannot mix defaultValue with onChange

✅ **Answer:** B.

Uncontrolled components: useRef usage

11. Which statement best describes an **uncontrolled component** in React?

A. A component that keeps form data in React state.
B. A component that uses the DOM itself to store form data.
C. A component that uses the useState hook to track input values.
D. A component that automatically resets inputs on submit.

✅ **Answer:** B

---

12. In an uncontrolled component, how is the current value of an input typically accessed?

A. Using onChange and useState.
B. By reading e.target.value in onChange.
C. Using a ref via useRef and reading ref.current.value.
D. By directly modifying the value prop.

✅ **Answer:** C

---

13. What will be the output of the following code when the button is clicked?

function Form() {

```
 const inputRef = React.useRef();
 return (
  <>
   <input ref={inputRef} defaultValue="Hello" />
   <button onClick={() => console.log(inputRef.current.value)}>Show</button>
  </>
 );
}
```

A. Logs nothing
B. Logs Hello initially and current input value on subsequent clicks
C. Always logs an empty string
D. Throws an error

✅ **Answer:** B

---

14. Which of the following is **true** about useRef in uncontrolled components?

A. It re-renders the component when .current changes.
B. It preserves the same .current object between renders.
C. It behaves like useState internally.
D. It must always hold a DOM node reference.

✅ **Answer:** B

---

15. Why might one prefer uncontrolled components over controlled components?

A. They are easier to validate in real-time.
B. They reduce React re-renders for simple forms.
C. They integrate poorly with external DOM libraries.
D. They automatically sync with global state.

✅ **Answer:** B

---

16. What is the key difference between value and defaultValue in input elements?

A. value sets the uncontrolled input's live value.
B. defaultValue is for controlled components only.
C. defaultValue sets the initial value for uncontrolled inputs, while value is for controlled

inputs.
D. They behave identically in all cases.

✅ **Answer:** C

---

17. Given:

```
function MyForm() {
 const checkboxRef = React.useRef();
 function handleSubmit() {
  console.log(checkboxRef.current.checked);
 }
 return (
  <>
   <input type="checkbox" ref={checkboxRef} />
   <button onClick={handleSubmit}>Submit</button>
  </>
 );
}
```

What will be logged when the button is clicked?

A. The initial defaultChecked value
B. The current checked state of the checkbox
C. undefined
D. A DOM element

✅ **Answer:** B

---

18. Which line correctly sets the initial value of an uncontrolled input using useRef?

A. inputRef.value = "Hello";
B. <input value="Hello" ref={inputRef} />
C. <input defaultValue="Hello" ref={inputRef} />
D. inputRef.current.defaultValue = "Hello";

✅ **Answer:** C

---

19. What happens if you try to read inputRef.current.value before the first render?

A. It returns an empty string.
B. It returns undefined.
C. It throws a null reference error.
D. It logs the initial defaultValue.

✅ **Answer:** B

---

20. Which of the following is **not** a best practice when using uncontrolled components?

A. Using useRef to read input values on submit.
B. Combining useRef with defaultValue.
C. Using onChange handlers to set state for every keystroke.
D. Accessing form data via refs at submission time.

✅ **Answer:** C

Form Validation Basics: Required, Email, Password Match, Error & Blur Handling

21. What will be displayed when the user leaves a required input field blank and presses submit (with HTML5 validation enabled)?

<input type="text" required />

A. Form submits successfully
B. Browser shows “Please fill out this field”
C. Console logs an error
D. Input auto-fills “N/A”

✅ **Answer:** B

---

22. Identify the correct way to check if an email is valid in JavaScript:

A. if (email.includes("@"))
B. if (/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email))
C. if (email.endsWith(".com"))
D. if (email.startsWith("mail"))

✅ **Answer:** B

---

23. What will this code output when email = "usergmail.com"?

```
if (!email.includes("@")) {

  console.log("Invalid Email");
```

```
} else {
  console.log("Valid Email");
}
```

A. Valid Email
B. Invalid Email
C. Syntax Error
D. undefined

✅ **Answer:** B

---

24. Find the error in this React validation snippet:

```
<input value={email} onChange={(e) => setEmail(e.target.value)}>
```

A. Missing required attribute
B. Missing closing tag /
C. Should use onInput instead of onChange
D. value must be numeric

✅ **Answer:** B

---

25. If password = "abc123" and confirmPassword = "abc124", what happens?

```
if (password === confirmPassword) {
  console.log("Passwords match");
} else {
  console.log("Passwords do not match");
}
```

A. Passwords match
B. Passwords do not match
C. Code will throw TypeError
D. Nothing is printed

✅ **Answer:** B

---

26. What happens when the input loses focus in this React code?

```
<input onBlur={() => alert("Input lost focus")} />
```

A. The alert appears while typing
B. The alert appears when input loses focus

C. The alert never appears
D. Syntax Error
✅ **Answer:** B

---

27. Find the output for this code when the email input is empty:

```
const email = "";

if (!email) console.log("Email required");

else console.log("OK");
```

A. OK
B. Email required
C. undefined
D. null
✅ **Answer:** B

---

28. Which correction makes this code valid?

```
if (email === "") {

 setError("Email is required");

else {

 setError("");

}
```

A. Replace else with else if
B. Add missing closing brace before else
C. Add semicolon after if
D. Remove setError
✅ **Answer:** B

---

29. Which event is best for validating an input after the user finishes typing?

A. onChange
B. onClick
C. onBlur
D. onInput
✅ **Answer:** C

---

30. What will be logged for password = "Test123", confirmPassword = "Test123"?

```
if (password !== confirmPassword)
  console.log("Mismatch");
else
  console.log("Valid");
```

A. Mismatch
B. Valid
C. undefined
D. Error
✅ **Answer:** B

Build Login/Register Form with Validation & Error Display

31. What will this code log when both inputs are empty and form is submitted?

```
if (!email || !password) console.log("All fields required");
else console.log("Submitted");
```

A. Submitted
B. All fields required
C. Nothing
D. Error
✅ **Answer:** B

---

32. What is the problem in this React code?

```
<button onClick={handleSubmit()}>Submit</button>
```

A. handleSubmit is never called
B. handleSubmit runs immediately instead of on click
C. Button cannot have parentheses
D. JSX doesn't support buttons
✅ **Answer:** B

---

33. If API returns status 401 in a login form, what should the app show?

A. Reload page
B. Show inline message "Invalid credentials"
C. Show "Form submitted successfully"

D. Reset the form

✅ **Answer:** B

---

34. What will happen when a user types in this controlled component?

<input type="text" value={username} />

A. User can type freely
B. Input is read-only since onChange is missing
C. Syntax Error
D. It clears after each keystroke

✅ **Answer:** B

---

35. Find the expected console output:

const user = { email: "abc@gmail.com", password: "" };

if (!user.password) console.log("Password missing");

else console.log("Login success");

A. Login success
B. Password missing
C. Syntax error
D. Nothing

✅ **Answer:** B

---

36. Which change will fix this React snippet?

<form onSubmit={handleSubmit}>

<button type="Submit">Submit</button>

</form>

A. Change type to lowercase "submit"
B. Move button outside form
C. Add required attribute
D. Wrap inside a

✅ **Answer:** A

---

37. What will be displayed for this validation?

const pass = "Admin@1";

```
if (pass.length >= 8 && /[A-Z]/.test(pass))
  console.log("Strong Password");
else
  console.log("Weak Password");
```

A. Strong Password
B. Weak Password
C. Error
D. No output

✅ **Answer:** B *(only 7 chars: fails length check)*

---

38. What is the output?

```
const email = "test@domain.com";
console.log(email.endsWith(".com") ? "Valid" : "Invalid");
```

A. Valid
B. Invalid
C. undefined
D. Syntax Error

✅ **Answer:** A

---

39. Which statement best describes this pattern?

```
const [formData, setFormData] = useState({ email: "", password: "" });
<input name="email" onChange={handleChange} />
```

A. Controlled component with centralized state
B. Uncontrolled input field
C. Invalid JSX syntax
D. Direct DOM access

✅ **Answer:** A

---

40. What happens after successful registration with this code?

```
if (response.ok) {
  navigate("/login");
  alert("Registration successful!");
```

}

A. User stays on same page
B. Redirects to login page and shows success alert
C. Shows only alert, no redirect
D. Crashes app
✅ **Answer:** B